

Lab Guide 3

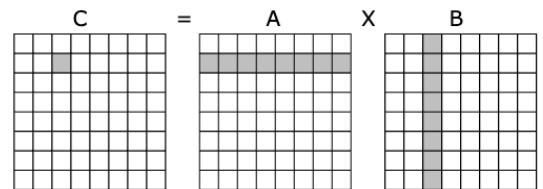
Vector Processing and Roofline Model

Objectives:

- Understand performance benefits and limitations of vector processing
- Use of the roofline model as a tool to identify hardware limitations and potential benefits of optimisations

Introduction

The peak performance of modern machines can only be attained when vector processing is explored. However, vector processing can only be applied to certain kinds of applications (e.g., data parallelism, where the same set of operations is applied to multiple elements) and requires *regular* data structures. Moreover, applications using vector processing consume more data per second, becoming more constrained by memory bandwidth. The roofline model is a visual tool that shows the peak performance and how memory bandwidth constrains the system's performance for a particular algorithm.



Exercise 1 – Compiler- based vectorisation

- a) Identification of limitations: The compiler cannot vectorise the “base” DOT matrix multiplication code. Identify two reasons for this (suggestion: look at the matrix data access patterns in the DOT code, namely, reads from A; reads from B and writes to C).
- b) Enabling vectorisation: The order of the loops can be changed without affecting the correctness of the results. Identify one (or more) order of the loops that support vectorisation. Implement the version that you selected and compile the code adding the options `-ftree-vectorize -msse4` (the second flag generates SSE4 vector instructions which supports vector operations of 128bits). How many elements are processed at once? What is the potential gain by using vector processing?
- c) Measure and discuss the results: Complete the table and comment the results (is the performance gain what was expected? Why?).

N	Version	Time	CPI	#I (inst_retired.any)
512	base_v()			
	vect() double			
	vect() float			

- d) Change the matrix data type from `double` to `float`. What is the expected gain of this change? Complete the table and explain the results.
- e) (*) Vectorisation fine-tuning: In order to further improve vectorisation you can compile with `-mavx` to use vector operations on 256bit vectors. However to take advantage of this vector size the data should be aligned at 32-byte memory addresses (Why?). One way to enforce this alignment in `gcc` is to declare the vector statically with the aligned attribute (e.g., `double A[size*size] __attribute__((aligned (32)))`; etc.).

